Birla Institute of Technology and Science, Pilani

Work Integrated Learning Programmes Division

M. Tech. in AI & ML

I Semester 2025-2026

Mid-Semester Test
(EC2 - Regular)

Course Number	AIMLCZG511
Course Name	DEEP NEURAL NETWORK
Nature of Exam	Closed Book
Weight-age for grading	30
Total Marks	100
Duration	2 hrs

# Pages	19
# Questions	5

---

1. (a) Consider a perceptron with 2 input features. Current weights are:

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.3 \\ -0.4 \end{bmatrix}$$

where w_0 is the bias, w_1 is the weight for feature x_1, and w_2 is the weight for feature x_2.

A training example has features $x_1 = 1, x_2 = 2$ with true label $y = 1$. Learning rate $\eta = 0.1$. The step activation function outputs 1 if $z \geq 0$, otherwise outputs 0.

Compute: (i) weighted sum z, (ii) predicted output $\hat{y}$ using step activation, (iii) if the example is misclassified, calculate the updated weights using the perceptron learning rule for each weight (note: treat bias w_0 as having input $x_0 = 1$). [6 marks]

(b) A spam classifier perceptron has learned weights $\mathbf{w} = [0.2, 0.8, 0.9, -0.5]^T$ (bias, suspicious_words, links, length).

i. Which feature most strongly indicates spam? Explain using weight magnitudes. [2 marks]

ii. The model achieves 75% accuracy but stops improving with more training. Explain what this reveals about the data's linear separability and the perceptron's limitations. [4 marks]

(c) A company is building a perceptron-based classifier to categorize customer reviews as positive or negative. They have collected 500 training reviews and are comparing two approaches:

Approach A: Uses 5 carefully selected features: count of positive sentiment words, count of negative sentiment words, presence of exclamation marks, review length (word count), and star rating (1-5).

Approach B: Uses 50 features representing the counts of the 50 most frequently occurring words across all training reviews.

Both approaches achieve 78% accuracy on the training set. Evaluate which approach is more likely to generalize better when deployed on new customer reviews. In your evaluation, consider the risk of overfitting given the limited training data and the interpretability of the learned decision boundary. [3 marks]

(d) Complete the Python code for perceptron training. For the following Python code, write only the code/expression that should replace each blank (Blank 1, Blank 2, etc.). Do not write the entire program. [5 marks]

```python
import numpy as np

def perceptron_train(X, y, learning_rate=0.1, epochs=10):
    """X: (N x d) with bias, y: (N,) labels {0,1}"""
    N, d = X.shape
    weights = np.zeros(d)

    for epoch in range(epochs):
        for i in range(N):
            z = ____________  # Blank 1
            y_pred = 1 if z >= 0 else 0

            if y_pred != y[i]:
                error = ____________  # Blank 2
                weights = weights + ____________  # Blank 3
    return weights

# Test
X = np.array([[1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]])
y = np.array([0, 0, 0, 1])  # AND gate
w = perceptron_train(X, y)
preds = ____________  # Blank 4
acc = ____________  # Blank 5
```

2. (a) A linear regression model predicts house price (in \$1000s) from area (in 100s of sq ft). The training data is:

Bias	Area	Price
1	10	150
1	20	250

Current weights are bias $w_0 = 50$, area weight $w_1 = 8$ and learning rate $\eta = 0.01$. Perform one batch gradient descent iteration: (i) compute predictions for both examples, (ii) calculate MSE loss, (iii) compute gradient, (iv) update weights. [6 marks]

(b) Complete the code for batch gradient descent. For the following Python code, write only the code/expression that should replace each blank (Blank 1, Blank 2, etc.). Do not write the entire program. [5 mark]

```python
import numpy as np

def train_linear(X, y, lr=0.01, epochs=100):
    """X: (N x d) with bias, y: (N,) targets"""
    N, d = X.shape
    weights = np.random.randn(d) * 0.01

    for epoch in range(epochs):
        y_pred = ____________  # Blank 1
        loss = ____________  # Blank 2
        error = y_pred - y
        gradient = ____________  # Blank 3
        weights = ____________  # Blank 4

        if epoch % 20 == 0:
            print(f"Epoch {epoch}, Loss: {loss:.4f}")

    return weights

# Usage
X = np.array([[1, 1], [1, 2], [1, 3]])
y = np.array([2, 4, 5])
w = ____________  # Blank 5
```

(c) A linear regression model predicts annual salary (in dollars) based on years of education and years of work experience. The learned weights are $w_0 = 30000$ (bias), $w_1 = 2500$ (education coefficient), $w_2 = 3000$ (experience coefficient).

i. Calculate the predicted salary for a person with 4 years of education and 2 years of work experience. Show your calculation. Discuss whether this predicted salary seems reasonable for an entry-level position with these qualifications. [3 marks]

ii. The model has a Root Mean Squared Error (RMSE) of \$2,236 on the test set. This RMSE was calculated from MSE = 5,000,000.. For salary predictions in a typical job market, is an average prediction error of \$2,236 considered acceptable? Explain your reasoning considering typical salary ranges. [3 marks]

(d) Two linear regression models are trained to predict apartment rent based on area (in square feet). Both models have identical parameters, make identical predictions, and achieve the same MSE on test data. The only difference is in the training process:

Model A: Trained using batch gradient descent on original features (area values range from 400 to 2000 sq ft).

Model B: Trained using batch gradient descent on normalized features (z-score normalization was applied to transform area values before training).

Evaluate which model would train faster (converge in fewer epochs) using gradient descent. Explain your answer with reference to the shape of the error surface and how feature scaling affects gradient descent convergence. [3 marks]

3. (a) A binary classifier for loan approval uses logistic regression with sigmoid activation. The current weights are $w_0 = 0$ (bias), $w_1 = 0.6$ (credit score weight), $w_2 = 0.8$ (income weight). The learning rate is $\eta = 0.1$. Process one training example with the following values: credit score $= 0.7$, income $= 0.5$, and true label approve $= 1$.

Calculate: (i) The weighted sum z (ii) the predicted probability using the sigmoid activation function (iii) the gradient for this single training example where $y = 1$ is the true label. (iv) The updated weights using gradient descent. [6 marks]

(b) Complete the mini-batch logistic regression code. For the following Python code, write only the code/expression that should replace each blank (Blank 1, Blank 2, etc.). Do not write the entire program. [5 mark]

```python
import numpy as np

def sigmoid(z):
    return ___________  # Blank 1

def train_logistic(X, y, batch_size=32, lr=0.01, epochs=100):
    """X: (N x d), y: (N,) binary labels"""
    N, d = X.shape
    weights = np.zeros(d)

    for epoch in range(epochs):
        indices = np.random.permutation(N)
        for i in range(0, N, batch_size):
            idx = indices[i:i+batch_size]
            X_batch, y_batch = X[idx], y[idx]

            z = np.dot(X_batch, weights)
            y_pred = ___________  # Blank 2
            gradient = ___________  # Blank 3
            weights = ___________  # Blank 4

    return weights

def predict(X, weights):
    return ___________  # Blank 5
```

(c) A medical diagnostic system uses logistic regression to detect a disease based on blood test results. The system was trained and tested on 1000 patients: 50 have the disease (positive class) and 950 are healthy (negative class). The model uses a classification threshold of 0.5 (predict disease if $\hat{y} \geq 0.5$).

The model's performance is:

- True Positives (correctly detected disease): 40 out of 50 diseased patients
- False Negatives (missed disease cases): 10 out of 50 diseased patients
- False Positives (false alarms): 95 out of 950 healthy patients incorrectly flagged as diseased
- True Negatives (correctly identified as healthy): 855 out of 950 healthy patients

This gives a recall (sensitivity) of 80% for disease detection.

i. Calculate the overall accuracy. Then explain why an accuracy of 89% is misleading for this highly imbalanced problem (where 95% of patients are healthy). What would a naive classifier that always predicts "healthy" achieve? [3 marks]

ii. For life-threatening disease detection, explain why recall (sensitivity) is more critical than precision for the positive class. If the classification threshold is lowered from 0.5 to 0.3 (making the model more likely to predict "disease"), explain how this would affect: (a) false positives, and (b) false negatives. [3 marks]

(d) A credit card company is deploying a fraud detection system using logistic regression. Their transaction dataset contains 10,000 transactions: 9,500 are legitimate and 500 are fraudulent. Two models have been trained and tested:

Model A: Uses 3 basic features (transaction amount, location, time of day). Achieves 95% overall accuracy and detects 60% of fraudulent transactions (300 out of 500 frauds caught).

Model B: Uses 20 features (including merchant category, device ID, purchase history patterns, etc.). Achieves 96% overall accuracy and detects 75% of fraudulent transactions (375 out of 500 frauds caught).

The company's cost analysis shows: Each missed fraud costs \$100 in losses, and each transaction flagged for investigation costs \$10 in manual review expenses.

Evaluate which model the company should deploy. In your evaluation, consider: (i) the business impact of the 15% improvement in fraud detection (75 additional frauds caught), (ii) the modest 1% improvement in overall accuracy, and (iii) whether the added complexity of Model B (20 features vs 3 features) is justified by the performance gains. [3 marks]

4. (a) A 3-class sentiment classifier uses softmax regression to categorize text reviews. The three sentiment classes are: negative (class 0), neutral (class 1), and positive (class 2). For a particular review, the model produces the following logits (pre-activation values). The true sentiment for this review is "neutral" (class 1), which is represented as the one-hot encoded vector.

$$\mathbf{z} = \begin{bmatrix} 2.0 \\ 1.0 \\ 0.5 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Calculate:

i. The softmax probabilities for all three classes. Show your calculation for at least one class, including the sum in the denominator. [2 marks]

ii. The Categorical Cross-Entropy (CCE) loss. [2 marks]

$$L = -\sum_{k=0}^{2} y_k \log(\hat{y}_k)$$

iii. Identify which class the model predicts. [1 mark]

iv. State whether the model's prediction is correct by comparing the predicted class with the true label. [1 mark]

(b) Complete the mini-batch softmax training code. For the following Python code, write only the code/expression that should replace each blank (Blank 1, Blank 2, etc.). Do not write the entire program. [5 marks]

```python
import numpy as np

def softmax(Z):
    """Z: (N, K) logits. Returns: (N, K) probabilities"""
    exp_Z = np.exp(Z - np.max(Z, axis=1, keepdims=True))
    return ___________  # Blank 1

def train_softmax(X, Y, K, batch_size=32, lr=0.01, epochs=100):
    """X: (N, d), Y: (N, K) one-hot labels"""
    N, d = X.shape
    W = np.random.randn(d, K) * 0.01

    for epoch in range(epochs):
        indices = np.random.permutation(N)
        for i in range(0, N, batch_size):
            idx = indices[i:i+batch_size]
            X_batch, Y_batch = X[idx], Y[idx]

            Z = ___________  # Blank 2
            Y_pred = softmax(Z)
            gradient = ___________  # Blank 3
            W = ___________  # Blank 4

    return W

def predict(X, W):
    Z = np.dot(X, W)
    return ___________  # Blank 5
```

(c) An image classifier for 3 animal types (cat, dog, bird) is trained and evaluated. The confusion matrix on the test set is:

	Pred Cat	**Pred Dog**	**Pred Bird**	**Total**
True Cat	280	15	5	300
True Dog	20	350	30	400
True Bird	10	40	250	300
Total	310	405	285	1000

i. Calculate precision and recall for the "Bird" class. Clearly identify the values of True Positives (TP), False Positives (FP), and False Negatives (FN) from the confusion matrix. [3 marks]

ii. Most bird misclassifications are predicted as "Dog" (40 cases out of 50 total bird errors). What does this confusion pattern suggest about the features the model has learned? Does the 88% overall accuracy guarantee good performance for each individual class? [3 marks]

(d) A hospital is developing a chest X-ray classifier to diagnose 5 conditions: pneumonia (8000 training images), tuberculosis (150 images), COVID-19 (200 images), lung cancer (180 images), and healthy (7000 images). The dataset is severely imbalanced. Two trained models are being compared for deployment:

Model A: Achieves 92% overall accuracy across all 5 classes, but only 45% recall for tuberculosis detection (misses more than half of TB cases).

Model B: Achieves 85% overall accuracy, but has 78% recall for tuberculosis detection (catches most TB cases).

Evaluate which model should be deployed in the hospital. In your evaluation, consider: the clinical consequences of missing tuberculosis cases (a serious and contagious disease), whether the 7% drop in overall accuracy is acceptable given the improved TB detection, and the ethical implications of optimizing for overall accuracy versus per-disease performance in medical diagnosis. [3 marks]

5. (a) Perform forward propagation through a 2-layer Deep Feedforward Neural Network for binary classification.

Network Architecture:

- Input layer: 2 features
- Hidden layer: 2 neurons with ReLU activation
- Output layer: 1 neuron with sigmoid activation

Network Parameters:

$$\mathbf{W}^{[1]} = \begin{bmatrix} 1.0 & 0.5 \\ -0.5 & 1.5 \end{bmatrix}, \qquad \mathbf{b}^{[1]} = \begin{bmatrix} 0.2 \\ -0.3 \end{bmatrix} \qquad \mathbf{w}^{[2]} = \begin{bmatrix} 1.2 \\ 0.8 \end{bmatrix}^T, \qquad b^{[2]} = 0.1$$

Input and Label: Input vector: $\mathbf{x} = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}$, True label: $y = 1$

Calculate: (i) Pre-activation values for hidden layer. (ii) Hidden layer activations after applying ReLU. (iii) Output prediction. (iv) Binary Cross-Entropy loss where $y = 1$. [6 marks]

(b) Complete the 2-layer DFNN code. For the following Python code, write only the code/expression that should replace each blank (Blank 1, Blank 2, etc.). Do not write the entire program. [5 mark]

```python
import numpy as np

def relu(Z):
    return ____________   # Blank 1

def sigmoid(Z):
    return 1 / (1 + np.exp(-Z))

def forward(X, W1, b1, W2, b2):
    Z1 = np.dot(X, W1) + b1
    A1 = ____________   # Blank 2
    Z2 = np.dot(A1, W2) + b2
    A2 = sigmoid(Z2)
    return A2, (Z1, A1, Z2, A2)

def backward(X, Y, cache, W2):
    Z1, A1, Z2, A2 = cache
    N = X.shape[0]

    dZ2 = A2 - Y
    dW2 = ____________   # Blank 3
    dA1 = ____________   # Blank 4
    dZ1 = dA1 * (Z1 > 0)
    dW1 = ____________   # Blank 5

    return dW1, dW2
```

(c) Design a Deep Feedforward Neural Network for sentiment classification with the following specifications:

- Input layer: 1000 features (word counts from text)
- Output layer: 5 classes (sentiment categories)
- Training data: 50,000 samples

Design and justify your network architecture by addressing the following:

i. Specify the complete architecture: How many neurons in the input layer? How many hidden layers will you use and how many neurons in each hidden layer? How many neurons in the output layer? [1 mark]

ii. Select appropriate activation functions: What activation function will you use for each layer? Justify your choices based on the classification task. [1 mark]

iii. Choose the loss function: What loss function is appropriate for this problem? Explain why this loss function is suitable. [1 mark]

iv. Specify the learning algorithm: Will you use batch gradient descent, stochastic gradient descent (SGD), or mini-batch gradient descent? Justify your choice. [1 mark]

v. Calculate total parameters: Using your proposed architecture, calculate the total number of trainable parameters. Given 50,000 training samples, does your parameter count seem reasonable to avoid overfitting? Explain. [2 marks]

(d) A company must select a Deep Feedforward Neural Network architecture for production deployment. The application is image classification on mobile devices with 10 output classes. The system must process 1 million images per day on smartphones with limited computational resources. Two architectures have been trained and evaluated:

Architecture A: $[256 \rightarrow 128 \rightarrow 64 \rightarrow 10]$ (3 hidden layers, gradually decreasing width)

- Total parameters: 50,000
- Test accuracy: 87%
- Inference time: 5 milliseconds per image
- Memory requirement: 200 MB

Architecture B: $[256 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10]$ (5 hidden layers, wider network)

- Total parameters: 850,000
- Test accuracy: 89%
- Inference time: 45 milliseconds per image
- Memory requirement: 3.4 GB

Evaluate which architecture the company should deploy for mobile devices. In your evaluation, consider: (i) Whether the 2% accuracy improvement (87% vs 89%) justifies the 9× slower inference speed (5ms vs 45ms). At 1 million images per day, calculate or estimate the total processing time difference. (ii) Mobile device constraints: typical smartphones have 4-6 GB total RAM (shared with OS and other apps) and battery life is critical for user experience. (iii) Would your recommendation change if the deployment target were cloud servers with unlimited computational resources instead of mobile devices? [3 marks]

Answer Key

1. (a) **Solution:**

   (i) Weighted sum calculation:

$$\begin{aligned} z &= w_0 + w_1 x_1 + w_2 x_2 \\ &= 0.5 + 0.3(1) + (-0.4)(2) = 0.0 \end{aligned}$$

   (ii) Predicted output using step activation: Since $z = 0.0 \geq 0$, we have $\hat{y} = 1$.

   (iii) Weight update: The example is correctly classified since $\hat{y} = 1 = y$. Therefore, no weight update is needed. The weights remain:

$$\mathbf{w}^{new} = \begin{bmatrix} 0.5 \\ 0.3 \\ -0.4 \end{bmatrix}$$

   (b) **Solution:**

   Blank 1: `np.dot(X[i], weights)`

   Blank 2: `y[i] - y_pred`

   Blank 3: `learning_rate * error * X[i]`

   Blank 4: `(np.dot(X, w) >= 0).astype(int)`

   Blank 5: `np.mean(preds == y)`

   (c) **Solution:**

   (i) The feature "links" (number of links) most strongly indicates spam with weight magnitude $|0.9| = 0.9$. This is the largest absolute weight value among the features.

   (ii) The 75% accuracy plateau indicates that the data is not perfectly linearly separable. The fact that accuracy stops improving suggests approximately 25% of the data lies on the wrong side of any possible linear decision boundary. The perceptron algorithm cannot learn non-linear decision boundaries that might be needed to perfectly separate spam from non-spam emails. The remaining 25% of errors likely require non-linear features or a more complex model (like a multi-layer neural network) to classify correctly.

   (d) **Solution:**

   Approach A is more likely to generalize better.

   Reasoning: With only 500 training reviews, Approach B using 50 features has a much higher risk of overfitting. The ratio of training samples to features is only 10:1 for Approach B, whereas Approach A has a 100:1 ratio.

2. (a) **Solution:**

(i) Predictions:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \begin{bmatrix} 1 & 10 \\ 1 & 20 \end{bmatrix} \begin{bmatrix} 50 \\ 8 \end{bmatrix} = \begin{bmatrix} 50 + 80 \\ 50 + 160 \end{bmatrix} = \begin{bmatrix} 130 \\ 210 \end{bmatrix}$$

(ii) MSE loss:

$$J = \frac{1}{2N} \sum_{i=1}^{N} (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{4}[(130 - 150)^2 + (210 - 250)^2] = 500$$

(iii) Gradient:

$$\nabla J = \frac{1}{N} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y}) = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 10 & 20 \end{bmatrix} \begin{bmatrix} 130 - 150 \\ 210 - 250 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 10 & 20 \end{bmatrix} \begin{bmatrix} -20 \\ -40 \end{bmatrix} = \begin{bmatrix} -30 \\ -500 \end{bmatrix}$$

(iv) Updated weights:

$$\mathbf{w}^{new} = \mathbf{w} - \eta \nabla J = \begin{bmatrix} 50 \\ 8 \end{bmatrix} - 0.01 \begin{bmatrix} -30 \\ -500 \end{bmatrix} = \begin{bmatrix} 50.3 \\ 13 \end{bmatrix}$$

(b) **Solution:**

Blank 1: `np.dot(X, weights)`

Blank 2: `np.mean((y_pred - y)**2) / 2` or `np.sum((y_pred - y)**2) / (2*N)`

Blank 3: `np.dot(X.T, error) / N`

Blank 4: `weights - lr * gradient`

Blank 5: `train_linear(X, y, lr=0.01, epochs=100)`

(c) **Solution:**

(i) Predicted salary calculation:

$$\hat{y} = w_0 + w_1 \times \text{education} + w_2 \times \text{experience} = 30000 + 2500(4) + 3000(2) = 46000$$

The prediction appears realistic for typical entry-level professional roles.

(ii) An RMSE of 2,236 means the model's predictions are typically off by about 2,236. For salary predictions, this is quite good. If typical salaries in the dataset range from 30,000 to 100,000, then a 2,236 error represents only about 3-7% relative error for most predictions. This level of accuracy would be acceptable

(d) **Solution:**

Model B (normalized features) trains faster via gradient descent.

Reasoning: Feature normalization (z-score) transforms all features to have similar scales (mean=0, std=1), creating a more spherical error surface. On a spherical surface, gradient descent can take more direct paths toward the minimum, requiring fewer iterations to converge. While both models achieve identical final results, Model B reaches that result in significantly fewer epochs due to better-conditioned optimization.

3. (a) **Solution:**

(i) Weighted sum:

$$z = w_0(1) + w_1(0.7) + w_2(0.5) = 0(1) + 0.6(0.7) + 0.8(0.5) = 0.82$$

(ii) Predicted probability:

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-0.82}} \approx 0.694$$

(iii) Gradient:

$$\nabla J = (\hat{y} - y)\mathbf{x} = (0.694 - 1)\begin{bmatrix} 1 \\ 0.7 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -0.306 \\ -0.214 \\ -0.153 \end{bmatrix}$$

(iv) Updated weights:

$$\mathbf{w}^{new} = \mathbf{w} - \eta \nabla J = \begin{bmatrix} 0 \\ 0.6 \\ 0.8 \end{bmatrix} - 0.1 \begin{bmatrix} -0.306 \\ -0.214 \\ -0.153 \end{bmatrix} = \begin{bmatrix} 0.0306 \\ 0.6214 \\ 0.8153 \end{bmatrix}$$

(b) **Solution:**

Blank 1: `1 / (1 + np.exp(-z))`

Blank 2: `sigmoid(z)`

Blank 3: `np.dot(X_batch.T, (y_pred - y_batch)) / len(y_batch)`

Blank 4: `weights - lr * gradient`

Blank 5: `(sigmoid(np.dot(X, weights)) >= 0.5).astype(int)`

(c) **Solution:**

(i) Overall accuracy calculation:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total}} = \frac{40 + 855}{1000} = 0.895 = 89.5\%$$

This 89.5% accuracy is misleading because the dataset is highly imbalanced (95% healthy). A naive classifier that always predicts "healthy" would achieve $\frac{950}{1000} = 95\%$ accuracy, which is even higher! The high accuracy primarily reflects the model's ability to correctly identify the majority class (healthy patients), but masks its poor performance on the critical minority class (diseased patients). For medical diagnostics, correctly identifying diseased patients is far more important than the overall accuracy metric suggests.

(ii) For life-threatening disease detection, recall (sensitivity) is more critical than precision because the cost of a false negative (missing a disease) is much higher than a false positive (unnecessary follow-up testing). Missing a disease diagnosis can lead to disease progression, complications, or death, while a false positive only results in additional testing and temporary patient anxiety.

If the threshold is lowered from 0.5 to 0.3:

False positives would increase: More healthy patients would be incorrectly flagged as diseased because the model becomes more "sensitive" to any indication of disease.

False negatives would decrease: Fewer diseased patients would be missed because the lower threshold makes it easier for the model to predict "disease."

This trade-off is acceptable in medical screening where catching all disease cases is prioritized over minimizing false alarms.

(d) **Solution:**

Model B should be deployed.

Reasoning: The 15% improvement in fraud detection (from 60% to 75%) means Model B catches 75 additional fraudulent transactions compared to Model A. At 100 loss per missed fraud, this represents 7,500 in prevented losses. The 1% improvement in overall accuracy is modest but the fraud detection improvement is substantial. While Model B uses more features (20 vs 3), increasing potential complexity, the business value of catching 75 more fraudulent transactions per 500 frauds far outweighs any concerns about model complexity. The additional features (merchant category, device ID, purchase history) capture important fraud patterns that basic features miss. For a credit card company, the financial benefit of better fraud detection justifies deploying the more complex but more effective Model B.

4. (a) **Solution:**

(i) Softmax probabilities:

$$\begin{aligned}
\text{Compute exponentials} \quad e^{z_0} &= e^{2.0} \approx 7.389 \\
e^{z_1} &= e^{1.0} \approx 2.718 \\
e^{z_2} &= e^{0.5} \approx 1.649 \\
\text{Sum of exponentials} \quad \sum_{j=0}^{2} e^{z_j} &= 7.389 + 2.718 + 1.649 = 11.756 \\
\text{Softmax probabilities} \quad \hat{y}_0 &= \frac{e^{2.0}}{11.756} = \frac{7.389}{11.756} \approx 0.628 \\
\hat{y}_1 &= \frac{e^{1.0}}{11.756} = \frac{2.718}{11.756} \approx 0.231 \\
\hat{y}_2 &= \frac{e^{0.5}}{11.756} = \frac{1.649}{11.756} \approx 0.140 \\
\hat{\mathbf{y}} &= \begin{bmatrix} 0.628 & 0.231 & 0.140 \end{bmatrix}^\top
\end{aligned}$$

(ii) CCE loss:

$$L = -\sum_{k=0}^{2} y_k \log(\hat{y}_k) = -(0 \cdot \log(0.628) + 1 \cdot \log(0.231) + 0 \cdot \log(0.140)) \approx 1.466$$

(iii) Predicted class: Class 0 (negative) has the highest probability (0.628).

(iv) The prediction is incorrect. The true label is class 1 (neutral), but the model predicts class 0 (negative).

(b) **Solution:**

Blank 1: `exp_Z / np.sum(exp_Z, axis=1, keepdims=True)`

Blank 2: `np.dot(X_batch, W)`

Blank 3: `np.dot(X_batch.T, (Y_pred - Y_batch)) / len(Y_batch)`

Blank 4: `W - lr * gradient`

Blank 5: `np.argmax(softmax(Z), axis=1)`

(c) **Solution:**

(i) For the "Bird" class:

From the confusion matrix:

True Positives (TP): 250 (correctly predicted as bird)

False Positives (FP): 285 - 250 = 35 (predicted as bird but actually cat or dog)

False Negatives (FN): 300 - 250 = 50 (actually bird but predicted as cat or dog)

Precision calculation:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{250}{250 + 35} = \frac{250}{285} \approx 0.877 = 87.7\%$$

Recall calculation:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{250}{250 + 50} = \frac{250}{300} \approx 0.833 = 83.3\%$$

(ii) The pattern of birds being misclassified as dogs (40 out of 50 bird errors) suggests the model has learned features that are shared between birds and dogs but distinguish

them from cats. This could indicate the model is focusing on features like: outdoor/-nature backgrounds, size variation, or movement patterns that birds and dogs share more than cats do. The model may be missing distinctive bird features like feathers, beaks, or wings.

The 88% overall accuracy does NOT guarantee good per-class performance. In this case, while overall accuracy is high, the bird class has only 83.3% recall, meaning nearly 17% of bird images are misclassified. Class imbalance (cat: 300, dog: 400, bird: 300) can lead to high overall accuracy even when minority or specific classes perform poorly.

(d) **Solution:**

Model B should be deployed.

Reasoning: In medical diagnosis, missing tuberculosis cases has severe clinical and public health consequences. TB is contagious, treatable when detected early, but potentially fatal if missed. Model A's 45% TB recall means it misses 55% of TB cases (82-83 out of 150), which is unacceptable for a serious infectious disease. Model B's 78% recall, while not perfect, catches most TB cases (117 out of 150), reducing missed diagnoses significantly.

The 7% drop in overall accuracy (from 92% to 85%) is acceptable because overall accuracy is dominated by the abundant classes (pneumonia: 8000, healthy: 7000). The critical metric for hospital deployment is per-disease performance, especially for rare but serious conditions like TB. Ethically, optimizing for overall accuracy at the expense of rare disease detection creates healthcare disparities where patients with less common conditions receive substandard care. Model B's balanced performance across all diseases, including rare ones, makes it the ethically and clinically appropriate choice.

5. (a) **Solution:**

(i) Hidden layer pre-activation:

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]T}\mathbf{x} + \mathbf{b}^{[1]} = \begin{bmatrix} 1.0 & -0.5 \\ 0.5 & 1.5 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix} + \begin{bmatrix} 0.2 \\ -0.3 \end{bmatrix} = \begin{bmatrix} 0.3 \\ 1.15 \end{bmatrix}$$

(ii) Hidden layer activations (ReLU):

$$\mathbf{a}^{[1]} = \max(0, \mathbf{z}^{[1]}) = \max\left(0, \begin{bmatrix} 0.3 \\ 1.15 \end{bmatrix}\right) = \begin{bmatrix} 0.3 \\ 1.15 \end{bmatrix}$$

(iii) Output prediction:

$$z^{[2]} = \mathbf{w}^{[2]}\mathbf{a}^{[1]} + b^{[2]} = \begin{bmatrix} 1.2 & 0.8 \end{bmatrix} \begin{bmatrix} 0.3 \\ 1.15 \end{bmatrix} + 0.1 = 1.38$$

$$\hat{y} = \sigma(z^{[2]}) = \frac{1}{1 + e^{-1.38}} \approx 0.799$$

(iv) BCE loss:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] = -[1 \cdot \log(0.799) + 0 \cdot \log(0.201)] \approx 0.224$$

(b) **Solution:**

Blank 1: `np.maximum(0, Z)`

Blank 2: `relu(Z1)`

Blank 3: `np.dot(A1.T, dZ2) / N`

Blank 4: `np.dot(dZ2, W2.T)`

Blank 5: `np.dot(X.T, dZ1) / N`

(c) **Solution:**

(i) Architecture specification:
Input layer: 1000 neurons (one for each word count feature)
Hidden layers: One hidden layer with 8 neurons (as specified: $n_1 = 8$)
Output layer: 5 neurons (one for each sentiment class)
Architecture: $[1000 \to 8 \to 5]$

(ii) Activation functions:
Hidden layer: ReLU activation ($f(z) = \max(0, z)$). ReLU is chosen because it addresses the vanishing gradient problem, enables faster training than sigmoid/tanh, and introduces non-linearity needed to learn complex patterns.
Output layer: Softmax activation ($\hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$). Softmax is required for multi-class classification because it converts raw logits into a probability distribution over the 5 classes, ensuring outputs sum to 1.

(iii) Loss function: Categorical Cross-Entropy (CCE): $L = -\sum_{k=1}^{5} y_k \log(\hat{y}_k)$ where $\mathbf{y}$ is the one-hot encoded true label. CCE is suitable for multi-class classification because it measures the dissimilarity between the predicted probability distribution and the true distribution, heavily penalizing confident wrong predictions.

(iv) Learning algorithm: Mini-batch gradient descent with batch size 32-64. Mini-batch is preferred over batch GD (which would be slow with 50,000 samples) and pure SGD (which would be noisy). Mini-batch balances computational efficiency with stable gradient estimates.

(v) Parameter calculation:

$$\text{Parameters} = (n_0 \times n_1 + n_1) + (n_1 \times n_2 + n_2) = (1000 \times 8 + 8) + (8 \times 5 + 5) = 8053$$

With 50,000 training samples and 8,053 parameters, the ratio is approximately 6.2:1 (samples to parameters). This is reasonable and should avoid overfitting. A common rule of thumb suggests having at least 5-10 times more training samples than parameters. With 8,053 parameters and 50,000 samples, the model has sufficient training data to learn generalizable patterns without overfitting.

(d) **Solution:**

Architecture A should be deployed for mobile devices.

Reasoning: (i) The 2% accuracy improvement (87% vs 89%) does not justify the 9× slower inference. At 1 million images per day: Architecture A requires $10^6 \times 5ms = 5 \times 10^6 ms \approx 1.4$ hours total processing time, while Architecture B requires $10^6 \times 45ms = 45 \times 10^6 ms = 12.5$ hours. On mobile devices, this massive time difference directly impacts battery life and user experience.

(ii) Architecture B's 3.4 GB memory requirement is problematic for mobile deployment. Typical smartphones have 4-6 GB RAM, but this is shared among the OS, background apps, and user applications. Consuming 3.4 GB for one model would cause memory pressure, app crashes, and poor user experience. Architecture A's 200 MB requirement is much more reasonable for mobile constraints. Additionally, the slower inference drains battery significantly faster.

(iii) The recommendation would change for cloud servers. With unlimited compute and memory resources, Architecture B's 2% accuracy improvement becomes more valuable. Cloud deployment eliminates battery concerns, and the 45ms inference time is still acceptable for most applications. The higher accuracy would improve user satisfaction, and the infrastructure costs of running the larger model are manageable at scale. Therefore, Architecture B would be preferred for cloud deployment where computational constraints are not limiting factors.